UNIVERSIDAD AUTÓNOMA DE ENTRE RÍOS
FACULTAD DE CIENCIAS Y TECNOLOGÍA

LICENCIATURA EN SISTEMAS DE INFORMACIÓN Y ANÁLISIS DE SISTEMAS

PROGRAMACIÓN ORIENTADA A OBJETOS

Trabajo Práctico Anual: “Juego por turnos piedra-papel-tijera”

Docentes:
-Emanuel Goette
-Leonardo Brambilla

Integrantes del grupo:
-Bifiger, Iara

-Gallay, Valentino

-Montani, Candela Itatí

-Villalba, Ezequiel

Comisión 1. Lunes

Año lectivo: 2026

INTRODUCCIÓN **3**
DESCRIPCIÓN GENERAL DEL SISTEMA **3**
DECISIONES DE DISEÑO Y CAMBIOS REALIZADOS **4**
Organización de las clases: 7
Clase Juego: 7
Clase Jugador: 8
Clase JugadorHumano: 8
Relación con otras clases: 9
Clase ContrincanteIA: 10
Relación con otras clases: 11
Clase IAAleatoria: 11
Clase Elemento: 13
Relación con otras clases: 13
Clase SistemaCombate: 14
Relación con otras clases: 15
Estructuras auxiliares: 16
Relación con otras clases: 16
Señales: 17
Relación con otras clases: 17
Principales responsabilidades: 18
Principales métodos: 18
Relación con otras clases 19
Enumerador TipoElemento 19
Enumerador Dificultad 19

link del repositorio:
https://github.com/CandeItati/TPAnualPOO-JuegodeCartas.git

INTRODUCCIÓN

En el siguiente archivo se detalla el diseño de un juego de combate por turnos inspirado en la similitud de Piedra–Papel–Tijera, utilizando conceptos vistos en Programación Orientada a Objetos como lo son las clases, herencias, polimorfismo, entre otros. La dinámica, sin profundizar demasiado en ella, trata de que a cada jugador, un humano vs. una IA, se les otorga un número específico de elementos, los cuales deben enfrentar, en base a decisiones, para generar una competencia entre los mismos, aplicando daños según una matriz de efectividad configurable entre tipos.

El sistema incorpora lo llamado “elementos mixtos” los cuales permiten una combinatoria entre elementos para alcanzar distintos ataques y defensas en la estrategia del juego, como así mismo solamente se incluyó la IA Aleatoria al juego, las dos IAs, la SuperIA y la IA estratégica ya se encuentran implementadas en esta última entrega

Como último paso introductorio, mencionamos la intención de incorporar una arquitectura flexible, evitando llegar a una lógica centralizada, haciendo más permisivo la futura extensión sin la necesidad de modificar la estructura principal del sistema.

DESCRIPCIÓN GENERAL DEL SISTEMA

El sistema consiste en un juego de combate basado en turnos de enfrentamientos entre elementos en rondas consecutivas. La mecánica general toma como referencia el juego Piedra–Papel–Tijera, utilizando una lógica parecida e incorporando el uso de Inteligencia Artificial y elementos de otra índole con distintas capacidades de competición.

La partida consta de dos participantes compitiendo: un jugador humano y un oponente controlado por una IA. Ambos reciben la misma cantidad de elementos a utilizar durante la partida. Cada carta representa una unidad capaz de atacar y recibir daño según su elemento.

Al comenzar la partida, el sistema genera y reparte automáticamente cinco cartas de diferentes elementos para cada jugador. Las cartas inician con un valor de energía (vida) del 100%, el cual va disminuyendo en el combate según las cartas a jugar. Las cartas pueden representar tanto un tipo elemental simple como un elemento mixto, los cuales se crean a partir de la combinación permitida entre dos cartas permitiendo que un elemento ataque con un tipo y se defienda con otro diferente.
En cada ronda, los jugadores deben seleccionar una carta, la cual es comparada con la del oponente, determinando el daño que cada una sufre. Para ello se utiliza una matriz configurable de efectividad entre tipos elementales. Esta matriz permite definir cuánto daño 2 produce cada tipo de ataque frente a cada tipo de defensa, evitando valores fijos dentro de la lógica principal del sistema y permitiendo modificar fácilmente las reglas del juego.

Al momento del juego, el humano debe realizar la selección de la carta selección manualmente, mientras que la IA lo hace automáticamente.

Durante el combate, el daño calculado reduce la vida de los elementos involucrados. Cuando la vida de un elemento alcanza el valor cero, el elemento queda fuera de combate y el jugador afectado debe seleccionar otro elemento disponible de su colección para continuar la partida.

Durante el combate, el daño calculado reduce la vida de los elementos involucrados. Cuando la vida de un elemento alcanza el valor cero, el elemento queda fuera de combate y el jugador afectado debe seleccionar otro elemento disponible de su colección para continuar la partida.

DECISIONES DE DISEÑO Y CAMBIOS REALIZADOS

Durante el desarrollo de la tercera se realizaron diversas modificaciones al diseño original con el objetivo de mejorar la organización del sistema, facilitar su mantenimiento y permitir la incorporación de nuevas funcionalidades sin alterar la arquitectura principal.
Una de las últimas decisiones de diseño implementadas en la entrega anterior fue modelar todos los elementos del juego en una única clase Elemento, evitando la creación de múltiples subclases para cada tipo elemental. En lugar de utilizar herencia para representar las distintas variantes, cada objeto almacena internamente un tipo de ataque (**tipoAtaque)** y un tipo de defensa (**tipoDefensa)**, lo que nos deja representar tanto elementos simples como elementos mixtos utilizando la misma estructura de datos. Por ejemplo, una carta posee un ataque y defensa del mismo tipo y una carta mixta combina distintos tipos elementales.

Gracias a este enfoque se evita la proliferación de clases con comportamientos similares, reduciendo la complejidad del modelo y facilitando su escalabilidad. La incorporación de nuevos elementos únicamente requiere agregar nuevos valores al enumerador correspondiente y definir sus relaciones dentro de la matriz de efectividad, sin necesidad de modificar la estructura general del sistema.
La clase Elemento fue diseñada con única responsabilidad: administrar el estado interno de cada carta durante la partida. Entre sus funciones se encuentran el almacenamiento de la vida, la identificación de sus tipos elementales y la gestión de su disponibilidad para combatir. Es por esta misma situación que la lógica encargada de calcular el daño no se encuentra dentro de esta misma clase, sino que se delegó a la clase **SistemaCombate**, haciendo que la separación de responsabilidades se apliquen de una mejor manera. Además, también fue incorporado el método **esMixto()**, el cual permite identificar de forma explícita aquellos elementos que combinan distintos tipos de ataque y defensa, facilitando futuras ampliaciones del sistema.

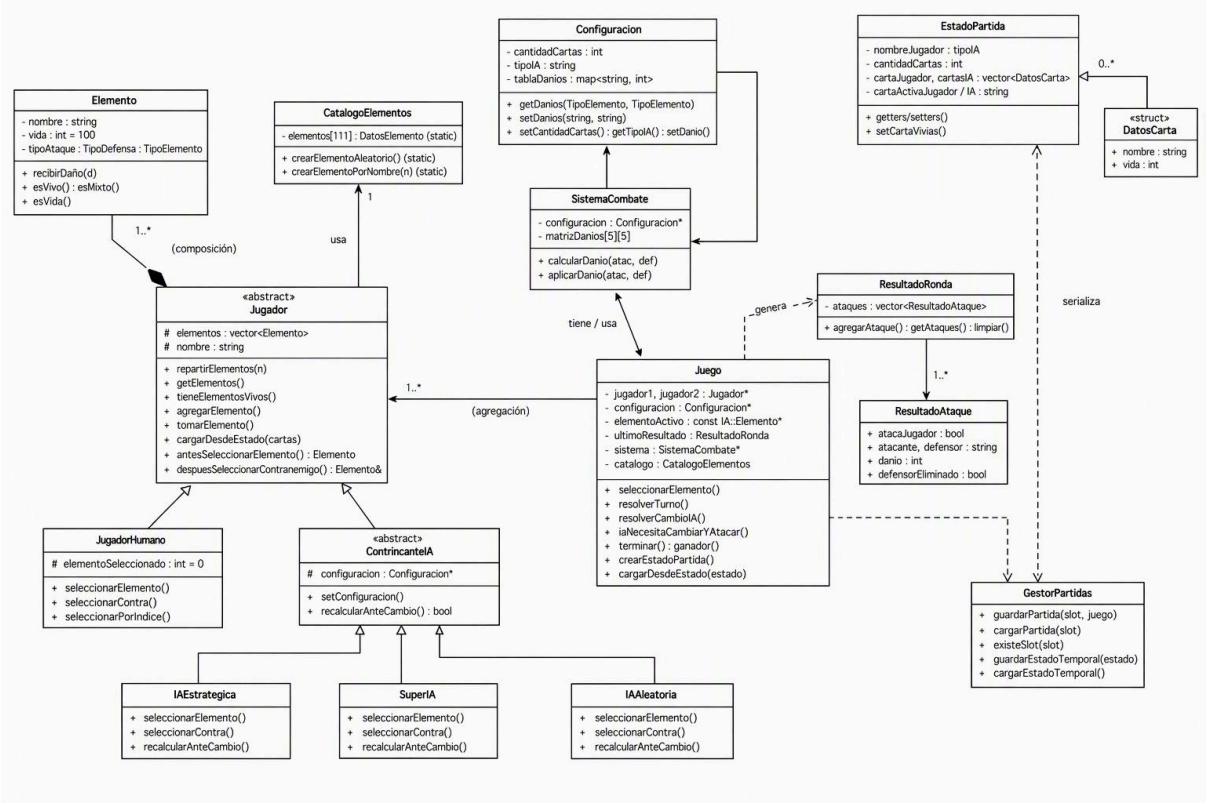
Respecto de las modificaciones incorporadas en esta tercera entrega, se realizaron clases como la clase de **configuración** donde allí se encuentra un archivo de texto el cual guarde la tabla de daños de cada carta, así mismo junto a ella se encuentra la clase **lectorconfiguracion,** esta misma se encarga como su nombre lo dice de leer el dicho archivo de texto.
Otra clase de archivos agregada fue la clase **gestorpartidas** la cual se encarga de ir guardando los datos realizados por la clase **estadoPartidas,** esta última clase va recopilando los datos de las partidas jugadas permitiendo.
Como también se implementaron clases como:**"resultadoataque, resultadoronda"** que fueron implementadas para una mejor experiencia para el jugador.
Y como anteultima implementación, se agregaron las dos IAs restantes **SuperIA y Estratégica**

Finalmente, el programa tiene dos ventanas "de nivel superior" que se reemplazan mutuamente: **MenuPrincipal** y **MainWindow** (la partida). Desde el menú se oculta **(hide())** el menú y se muestra una MainWindow nueva; desde la partida se crea un MenuPrincipal nuevo y se cierra la MainWindow **(close()).** No hay una ventana contenedora ni un *QStackedWidget*; el "estado de la aplicación" es simplemente cuál ventana está visible.
adaptándolo a una aplicación gráfica desarrollada con Qt sin alterar las responsabilidades principales de las clases ni el grado de desacoplamiento entre los distintos componentes del sistema.

también se han incorporado reglas a este proyecto las cuales son:

- **Mano inicial:** cada jugador recibe **N** cartas elegidas al azar (con reposición, por lo que puede haber repetidas) del catálogo de 11 elementos. N proviene de **cantidad_cartas** en *config.txt* (5 por defecto).
- **Vida:** toda carta nace con 100 de vida; la vida nunca baja de 0 (se satura en Elemento::recibirDanio).
- **Turno:** el humano elige una de sus cartas vivas y pulsa “ATACAR”. La IA responde eligiendo su carta según su estrategia. Se resuelve un intercambio: primero golpea el humano y, si la carta de la IA sigue viva, la IA contraataca.
- **Reposición de la IA:** si la carta de la IA muere con el golpe del humano, la IA elige una carta nueva y ataca de inmediato (no pierde su turno).
- **Reposición del humano:** si su carta muere, debe elegir otra antes de poder volver a atacar (el botón queda deshabilitado).
- **Fin de partida:** cuando algún bando se queda sin cartas con vida > 0. Se muestra una pantalla de Victoria o Derrota con estadísticas.
- **Tres IAs** de dificultad creciente (Aleatoria, Estratégica, Super) y una opción “SORTEO” que elige una al azar.
- **Guardado:** 5 ranuras (slots) de partida; además existe un guardado temporal para “Reintentar” una partida perdida con la mano inicial.

DIAGRAMA DE CLASES: UML



EXPLICACIÓN DE LAS CLASES PRINCIPALES

Organización de las clases:

Las clases del sistema fueron diseñadas siguiendo el principio de separación de responsabilidades. Cada una posee una función específica dentro de la arquitectura del juego, evitando concentrar múltiples tareas en un mismo componente. A continuación se describen las principales clases desarrolladas y el rol que cumplen dentro del sistema.

Clase Juego:

La clase **Juego** constituye el núcleo de la lógica del sistema y actúa como coordinadora de la partida. Su principal responsabilidad es mantener el estado actual del juego y coordinar la interacción entre los jugadores, el sistema de combate y la interfaz gráfica.

Durante la construcción del objeto, la clase recibe a ambos jugadores y les asigna automáticamente cinco elementos mediante el método **`repartirElementos()`.** Posteriormente administra la selección de cartas, coordina el desarrollo de cada enfrentamiento, determina cuándo un jugador debe cambiar de elemento, verifica la condición de fin de partida y genera los mensajes de estado que serán mostrados por la interfaz gráfica.

Es importante destacar que la clase **Juego** no implementa directamente el cálculo del daño ni contiene código relacionado con la representación gráfica. Estas responsabilidades son delegadas respectivamente a las clases **SistemaCombate** y **MainWindow**, manteniendo una clara separación entre la lógica del juego y la interfaz de usuario.

Entre sus principales métodos se encuentran:

- ***Juego(Jugador, Jugador***): inicializa la partida, recibe ambos jugadores y reparte automáticamente cinco elementos a cada uno.
- **seleccionarElemento(int)**: permite al jugador humano seleccionar una carta y solicitar la selección inicial de la IA.
- **resolverTurno()**: coordina el combate entre ambos elementos activos utilizando la clase SistemaCombate
- **resolverCambioIA()**: permite que la IA seleccione automáticamente un nuevo elemento cuando el anterior ha quedado fuera de combate.
- **cancelarSeleccion()**: elimina la selección actual del jugador para permitir elegir otra carta.
- **Término()**: verifica si alguno de los jugadores ya no posee elementos disponibles.
- **GanoJugador()**: determina si el jugador humano resultó vencedor.

- **GetMensajeEstado()**: devuelve el mensaje que debe mostrarse en la interfaz gráfica.

Relación con otras clases:

- Mantiene referencias a dos objetos **Jugador.**
- Utiliza un objeto **SistemaCombate** para resolver los enfrentamientos.
- Utiliza **CatalogoElementos** durante la creación inicial de las cartas.
- Es utilizada por **MainWindow**, que consulta su estado para actualizar la interfaz.

Clase Jugador:

La clase abstracta **Jugador** representa el comportamiento común de todos los participantes de la partida. Su principal responsabilidad consiste en administrar el conjunto de elementos pertenecientes al jugador y proporcionar las operaciones básicas necesarias para el desarrollo del combate.

La clase implementa la lógica común para repartir elementos y verificar si aún quedan cartas disponibles, mientras que delega en las clases derivadas la estrategia utilizada para seleccionar el elemento que participará del combate.

Entre sus métodos principales se encuentran:

- **repartirElementos()**: genera cinco elementos aleatorios utilizando **`CatalogoElementos`.**
- **getElementos()**: devuelve una referencia constante al conjunto de elementos del jugador.
- **tieneElementosVivos()**: verifica si el jugador todavía dispone de cartas activas.
- **seleccionarElemento()** *(virtual puro)*: define el mecanismo mediante el cual cada tipo de jugador elige una carta.
- **seleccionarContra()** *(virtual puro)*: permite implementar distintas estrategias para seleccionar el mejor elemento frente al enemigo.

Clase JugadorHumano:

La clase **JugadorHumano** representa al participante controlado por el usuario. Hereda de la clase abstracta **Jugador**, reutilizando toda la funcionalidad relacionada con la administración de elementos y especializando únicamente el mecanismo mediante el cual se selecciona la carta que participará en el combate.

A diferencia de las inteligencias artificiales, la selección del elemento no se realiza automáticamente, sino que depende de la interacción del usuario con la interfaz gráfica. Para ello, la clase mantiene internamente el índice del elemento actualmente seleccionado permitiendo que la lógica del juego consulte en cualquier momento cuál es la carta elegida.

De esta manera, la clase encapsula el comportamiento específico del jugador humano sin modificar la lógica común implementada en la clase **Jugador**, manteniendo una clara separación entre las distintas formas de selección de elementos.

Entre sus principales métodos se encuentran:

- **seleccionarElemento()**: devuelve una referencia al elemento actualmente seleccionado por el usuario.
- **seleccionarContra(const Elemento &enemigo)**: devuelve el elemento previamente seleccionado por el usuario. En esta implementación el elemento enemigo no influye en la decisión.
- **seleccionarPorIndice(int indice)**: actualiza el índice del elemento seleccionado según la elección realizada por el usuario.

Relación con otras clases:

- Hereda de la clase **Jugador**.
- Es utilizada por la clase **Juego** para obtener el elemento seleccionado por el usuario.
- Su selección depende de las acciones realizadas desde **MainWindow**.

Clase Configuración

La clase Configuración lee los parámetros del juego desde un archivo de texto ubicado en la raíz del proyecto, permitiendo modificar valores como la cantidad de elementos por jugador y el tipo de inteligencia artificial sin necesidad de recompilar el programa.

Resuelve la ubicación del archivo de forma independiente al directorio de ejecución, garantizando que siempre se busque junto al código fuente y no dentro de la carpeta de compilación. Si el archivo no existe o presenta líneas con formato incorrecto, la clase aplica valores por defecto y continúa su funcionamiento sin interrumpirse.

Métodos principales

El **constructor** inicializa los valores por defecto: cinco elementos por jugador y la inteligencia artificial aleatoria como tipo seleccionado.

El método **cargarArchivo** recibe opcionalmente el nombre del archivo, resuelve su ruta completa y lo abre; recorre cada línea, ignora comentarios y líneas vacías, interpreta cada par clave-valor y asigna los valores correspondientes.

Los métodos de consulta **getCantidadCartas, getTipoIA y getDanio** devuelven los valores configurados para que el resto del sistema los utilice sin conocer su origen.

Clase GestorPartidas

La clase GestorPartidas tiene como responsabilidad exclusiva el almacenamiento y la recuperación de las partidas, manteniendo esta función completamente separada de la lógica del juego y de la interfaz gráfica.

Admite un máximo de cinco partidas guardadas, cada una en su propio archivo, y garantiza que estos se creen y se lean siempre en la carpeta raíz del proyecto, nunca dentro del directorio de compilación, cumpliendo con el requisito de no entregar archivos generados en la carpeta build.

La clase provee además la información necesaria para que la pantalla de inicio muestra el estado de cada espacio de guardado sin exponer detalles internos del sistema de archivos.

GestorPartidas no hereda de ninguna clase. Se relaciona con la clase Juego, de la cual extrae el estado completo para guardarlo y a partir de la cual reconstruye la partida al cargarla.

No depende de clases de la interfaz gráfica, de modo que el guardado y la carga funcionan de forma independiente a la presentación. Utiliza la ruta raíz del proyecto definida en tiempo de compilación para ubicar los archivos de forma consistente sin importar desde dónde se ejecute el programa.

Clase `ContrincanteIA`:

La clase abstracta **ContrincanteIA** representa el comportamiento base de todos los jugadores controlados por inteligencia artificial. Hereda de la clase **Jugador**, reutilizando toda la funcionalidad relacionada con la administración de elementos y definiendo la estructura que deberán seguir las distintas estrategias de IA implementadas en el sistema.

Al tratarse de una clase abstracta, no implementa un algoritmo de selección concreto. En cambio, declara los métodos virtuales **seleccionarElemento()** y **seleccionarContra()**, permitiendo que cada inteligencia artificial defina su propia estrategia mediante polimorfismo.

Además, incorpora una instancia de la clase **SistemaCombate**, la cual podrá ser utilizada por futuras implementaciones para analizar enfrentamientos y tomar decisiones más elaboradas.

Entre sus principales métodos se encuentran:

- **seleccionarElemento()** *(virtual puro)*: define el mecanismo mediante el cual una IA selecciona el elemento que utilizará durante el combate.
- **seleccionarContra(const Elemento &enemigo)** *(virtual puro)*: permite implementar estrategias que consideren el elemento rival antes de realizar la selección.

Relación con otras clases:

- Hereda de la clase **Jugador**.
- Es la clase base de **IAAleatoria**, **IAEstrategica** y **SuperIA**.
- Contiene una instancia de **SistemaCombate** para facilitar el desarrollo de futuras estrategias.

Clase `IAAleatoria`:

La clase **IAAleatoria** implementa una estrategia de inteligencia artificial basada en la selección aleatoria de elementos. Hereda de **ContrincanteIA** y proporciona la primera implementación funcional del sistema de IA utilizada durante la segunda entrega del proyecto.

Su funcionamiento consiste en recorrer el conjunto de elementos disponibles, identificar aquellos que aún poseen vida y seleccionar uno de ellos de manera aleatoria. De esta forma se garantiza que la inteligencia artificial nunca elija una carta derrotada.

El método **seleccionarContra()** no realiza ningún análisis sobre el elemento enemigo. En esta implementación simplemente delega la decisión al método **seleccionarElemento()**, manteniendo un comportamiento completamente aleatorio.

Esta clase constituye una implementación sencilla que permitió validar el funcionamiento general del sistema y servirá como base para el desarrollo de estrategias más avanzadas en futuras versiones.

Entre sus principales métodos se encuentran:

- **seleccionarElemento():** selecciona aleatoriamente uno de los elementos que aún continúan con vida.
- **seleccionarContra(const Elemento &enemigo):** ignora las características del enemigo y realiza una selección aleatoria.

Clase `IAEstrategica` :

Estrategia de juego inteligente que extiende el comportamiento de `IAAleatoria`. A diferencia de esta última, no elige al azar: evalúa la ventaja de cada elemento contra el enemigo y selecciona el más conveniente. Si no hay información clara o todos los elementos son igual de ventajosos, recurre a una elección aleatoria.

Relaciones:

- Hereda públicamente de `IA` (clase base abstracta)
- Es la clase padre de `SuperIA`, que reutiliza su infraestructura y mejora su lógica de decisión
- Depende de `SistemaCombate` para consultar la tabla de efectividad de daños entre elementos
- Depende de **`GeneradorAleatorio`** para decisiones cuando hay empate o falta de información

Clase `SuperIA`:

Estrategia de juego avanzada que hereda de `IAEstrategica`. Su objetivo es optimizar el uso de los elementos: si puede rematar al enemigo con un elemento que sobreviva al ataque y sea el más débil disponible, lo elige para conservar los más fuertes para enfrentamientos posteriores. Si no puede rematar, se comporta igual que su clase padre.

Relaciones:

- Hereda públicamente de `IAEstrategica`
- Reutiliza métodos de la clase base: `indicesVivos()`, `elemento()`, `danioQueRecibiria()`, `danioQueHaria()`, `sortearEntre()`

`SuperIA(...)` Constructor: inicializa la parte de **`IAEstrategica`** con los parámetros recibidos

`tipo()` Devuelve el nombre identificador de la IA

`decidirIndice(enemigo)` Lógica de selección: busca rematadores → elige el más débil → si no hay, delega en la clase padre

`rematadores(enemigo)` Devuelve índices de elementos que sobreviven al ataque enemigo y lo eliminan al contraatacar

`losMasDebiles(candidatos)` Filtra de la lista recibida aquellos con la menor vida actual

Clase `Elemento`:

La clase **Elemento** representa una carta del juego y constituye la unidad básica de combate utilizada durante toda la partida. Cada objeto almacena toda la información necesaria para participar en un enfrentamiento, incluyendo su nombre, cantidad de vida, tipo de ataque y tipo de defensa.

Una de las principales decisiones de diseño consistió en representar tanto los elementos simples como los elementos mixtos mediante una única clase. Para ello, cada instancia mantiene de forma independiente un tipo de ataque y un tipo de defensa, permitiendo modelar distintas combinaciones elementales sin recurrir a múltiples subclases.

La responsabilidad de esta clase se limita exclusivamente a administrar su propio estado interno. Las reglas relacionadas con el cálculo del daño fueron delegadas a la clase **SistemaCombate**, manteniendo una adecuada separación de responsabilidades.

Entre sus métodos principales se encuentran:

- **obtenerNombre()**: devuelve el nombre del elemento.
- **obtenerAtaque()**: devuelve el tipo elemental utilizado al atacar.
- **obtenerDefensa()**: devuelve el tipo elemental utilizado para defenderse.
- **obtenerVida()**: devuelve la cantidad actual de vida.
- **recibirDanio()**: reduce la vida del elemento recibido en combate, evitando valores negativos.

- **estaVivo()**: indica si el elemento todavía puede participar en el combate.
- **esMixto()**: determina si el tipo de ataque y el tipo de defensa pertenecen a elementos diferentes.

Relación con otras clases:

- Es creada por **CatalogoElementos**.
- Es administrada por **Jugador**.
- Es utilizada por **SistemaCombate** para calcular los enfrentamientos.

A continuación se presentan ejemplos de configuraciones establecidas por nosotros:

Elemento Base	Tipo de Ataque y Tipo de Defensa
🔥 ***Fuego***	*FUEGO*
💧 ***Agua***	*AGUA*
🌱 ***Tierra***	*TIERRA*
⚡ ***Rayo***	*RAYO*
❄️ ***Hielo***	*HIELO*

Elemento Mixto	Tipo de Ataque	Tipo de Defensa
🌋 ***Lava***	*FUEGO*	*TIERRA*
💨 ***Vapor***	*FUEGO*	*AGUA*
⛈️ ***Tormenta***	*RAYO*	*AGUA*
🐊 ***Pantano***	*AGUA*	*TIERRA*
🧊 ***Glaciar***	*HIELO*	*AGUA*
💥 ***Plasma***	*RAYO*	*FUEGO*

Clase `SistemaCombate`:

La clase **SistemaCombate** concentra toda la lógica relacionada con la resolución de los enfrentamientos entre elementos. Su principal responsabilidad consiste en calcular y aplicar el daño correspondiente según los tipos elementales involucrados en el combate.

Durante su construcción inicializa una matriz bidimensional de daños, donde cada fila representa un tipo de ataque y cada columna un tipo de defensa. Los valores almacenados determinan la cantidad de daño que un elemento produce sobre otro de acuerdo con las reglas definidas para el juego.

Al centralizar esta lógica en una única clase se evita distribuir las reglas de combate en distintos componentes del sistema, facilitando el mantenimiento y permitiendo modificar el balance del juego sin alterar el resto de la aplicación.

Entre sus principales métodos se encuentran:

- **calcularDanio()**: consulta la matriz de efectividades y determina el daño correspondiente entre dos elementos.
- **aplicarDanio()**: aplica el daño calculado al elemento defensor.

Relación con otras clases:

- Utiliza objetos **Elemento**.
- Es utilizada por **Juego** para resolver todos los enfrentamientos.

A continuación se presenta un ejemplo de configuración de daño personalizado por nosotros:

Atacante \ Defensor	🔥 Fuego	💧 Agua	🌱 Tierra	⚡ Rayo	❄️ Hielo
🔥 Fuego	25%	15%	50%	30%	60%
💧 Agua	50%	25%	20%	15%	30%
🌱 Tierra	30%	50%	25%	60%	15%
⚡ Rayo	30%	60%	15%	25%	30%
❄️ Hielo	15%	30%	50%	30%	25%

Clase catalogoelementos:

La clase **CatalogoElementos** centraliza la información correspondiente a todos los elementos disponibles dentro del juego. Funciona como un catálogo estático desde el cual pueden generarse cartas de manera aleatoria durante el inicio de cada partida.

La información de cada elemento (nombre, tipo de ataque y tipo de defensa) se almacena en una estructura auxiliar denominada **DatosElemento**, organizada dentro de un arreglo estático. Esta decisión evita duplicar información en distintas partes del sistema y facilita la incorporación de nuevos elementos.

La generación de cartas se realiza mediante el método estático **`crearElementoAleatorio()`**, el cual selecciona un elemento al azar utilizando **`QRandomGenerator()`** y construye un nuevo objeto **Elemento** con la información correspondiente.

Entre sus principales métodos se encuentran:

- **crearElementoAleatorio()**: genera un nuevo elemento seleccionado aleatoriamente del catálogo.

Estructuras auxiliares:

- **DatosElemento**: almacena el nombre y los tipos elementales de cada carta.
- **elementos[]**: arreglo estático que contiene todos los elementos disponibles.

Relación con otras clases:

- Crea objetos **Elemento**.
- Es utilizada por **Jugador** para repartir los elementos al comenzar la partida.

Clase cartagraphicsistem:

La clase **CartaGraphicsItem** representa la visualización gráfica de una carta dentro del tablero de juego. Hereda de `QGraphicsObject`, permitiendo integrar cada carta dentro de una `QGraphicsScene` y aprovechar las capacidades gráficas y de eventos proporcionadas por el framework Qt.

Su responsabilidad consiste exclusivamente en representar visualmente un objeto de la clase **Elemento** y administrar toda la interacción gráfica asociada a él. Para ello almacena una copia de la información necesaria del elemento (nombre, vida, tipo de ataque y tipo de defensa) junto con distintos estados visuales utilizados durante la partida, como la selección, el combate, el efecto *hover* y la derrota.

Además de dibujar completamente la carta mediante el método `paint()`, esta clase incorpora animaciones de movimiento, efectos visuales, ampliación al pasar el cursor, sombras dinámicas y detección de clics sobre cada carta. De esta manera se desacopla completamente la representación gráfica de la lógica del juego, permitiendo que la clase **Juego** permanezca independiente de cualquier detalle relacionado con la interfaz.

Entre sus principales métodos se encuentran:

- **paint()**: dibuja completamente la carta, incluyendo marco, imagen, nombre, vida y tipos elementales.
- **boundingRect()**: define el área ocupada por la carta dentro de la escena gráfica.
- **moverA()**: realiza una animación de desplazamiento hacia una posición determinada.
- **actualizarVida()**: actualiza la vida mostrada en pantalla y marca la carta como derrotada cuando corresponde.
- **volverAFila()**: devuelve la carta a su posición original dentro de la mano del jugador.
- **establecerEnCombate()**: modifica el estado visual de la carta cuando participa en un combate.
- **hoverEnterEvent()** y **hoverLeaveEvent()**: administran los efectos visuales al posicionar el cursor sobre la carta.
- **mousePressEvent()**: detecta la selección realizada por el usuario y emite la señal correspondiente.

Señales:

- **clickeada(CartaGraphicsItem*)**: notifica a la interfaz qué carta fue seleccionada por el usuario.

Relación con otras clases:

- Representa gráficamente un objeto **Elemento**.
- Es utilizada por **MainWindow** para construir la escena del juego.
- Se integra dentro de una **QGraphicsScene** mediante la infraestructura gráfica de Qt.

Clase MainWindow:

La clase **MainWindow** constituye la ventana principal de la aplicación y representa la capa de presentación del sistema. Su función consiste en establecer la comunicación entre el usuario y la lógica del juego, administrando todos los componentes gráficos y coordinando las acciones realizadas sobre la interfaz.

Durante su inicialización crea los objetos necesarios para la partida (jugador humano, inteligencia artificial y clase **Juego**), configura la escena gráfica donde se representarán las cartas e incorpora todos los componentes visuales utilizados durante el desarrollo de la partida.

A medida que el usuario interactúa con la aplicación, la clase recibe los eventos generados por la interfaz, invoca los métodos correspondientes de la clase **Juego** y actualiza posteriormente la representación gráfica según el nuevo estado del sistema. De esta manera mantiene una clara separación entre la lógica del juego y la interfaz de usuario, evitando que las reglas de combate dependen de componentes gráficos.

Además de administrar la interacción principal, esta clase implementa animaciones, efectos visuales, indicadores de daño, movimiento de cartas, mensajes de combate y la pantalla de finalización de la partida.

Principales responsabilidades:

- Inicializar la interfaz gráfica.
- Crear los jugadores y la instancia de **Juego**.
- Construir la escena gráfica del tablero.
- Mostrar las cartas de ambos jugadores.
- Detectar la selección de cartas realizada por el usuario.
- Ejecutar los ataques solicitados por el jugador.
- Actualizar la representación visual del estado de la partida.
- Mostrar animaciones y efectos gráficos.
- Informar el resultado final de la partida.

Principales métodos:

Constructor MainWindow(): Inicializa completamente la aplicación, crea los jugadores, instancia la clase **Juego**, construye la escena gráfica y prepara la interfaz para comenzar la partida.

iniciarEscena(): Configura la `QGraphicsScene`, carga el tablero, crea el botón de ataque, incorpora el indicador "VS" y prepara todos los elementos gráficos necesarios.

ColocarManoJugador(): Genera las representaciones gráficas de todas las cartas pertenecientes al jugador humano.

colocarManoIA(): Genera las representaciones gráficas correspondientes a la inteligencia artificial.

onCartaJugadorClickeada(): Procesa la selección de una carta realizada por el usuario e informa dicha selección a la clase **Juego**.

onBtnAtacarClicked(): Solicita a la clase **Juego** la resolución completa del turno, actualiza la información mostrada en pantalla y presenta las animaciones correspondientes al combate.

resolverCambioDeCartaIA(): Gestiona el cambio automático de carta cuando la inteligencia artificial pierde el elemento activo durante un combate.

actualizarPantalla(): Sincroniza completamente la interfaz gráfica con el estado actual del modelo del juego, actualizando posiciones, vidas, cartas activas y controles disponibles.

mostrarDanioFlotante(): Genera una animación temporal que representa visualmente el daño recibido por una carta durante el combate.

mostrarFinDePartida(): Deshabilita la interacción con la interfaz y presenta el resultado final de la partida.

crearOverlayResultado(): Construye la ventana superpuesta utilizada para mostrar el resultado final del juego.

Relación con otras clases

- Utiliza la clase **Juego** como controlador principal de la partida.
- Crea objetos **JugadorHumano** e **IAAleatoria**.
- Administra todos los objetos **CartaGraphicsItem** presentes en la escena.
- Utiliza **QGraphicsScene** como contenedor gráfico del tablero.
- Interactúa con la interfaz generada mediante **Qt Designer** (`mainwindow.ui`).

Enumeradores del Sistema:

El sistema utiliza enumeradores para representar conjuntos de valores constantes relacionados con la lógica del juego. Esta decisión mejora la legibilidad del código, evita el uso de valores literales (*magic numbers*) y facilita el mantenimiento de la aplicación.

Enumerador TipoElemento

El enumerador **TipoElemento** representa los distintos tipos elementales disponibles dentro del juego. Cada elemento almacena un tipo de ataque y un tipo de defensa utilizando este enumerador, mientras que la clase **SistemaCombate** emplea dichos valores como índices para consultar la matriz de daño.

Actualmente se encuentran definidos los siguientes tipos elementales:

- Fuego
- Agua
- Tierra
- Rayo
- Hielo

La utilización de este enumerador permite incorporar nuevos tipos elementales modificando únicamente la definición del enumerador y la matriz de daño correspondiente, sin alterar la estructura general del sistema.

Enumerador Dificultad

El enumerador **Dificultad** representa los distintos niveles de inteligencia artificial previstos dentro del diseño del juego.

Los niveles contemplados son:

- Fácil
- Normal
- Difícil

Aunque en esta entrega únicamente se implementó la **IA Aleatoria**, la incorporación de este enumerador permite extender el sistema con nuevas estrategias de inteligencia artificial sin modificar la arquitectura general de la aplicación.

DECISIONES DE LA INTERFAZ GRÁFICA

La interfaz gráfica fue desarrollada utilizando el framework Qt, empleando principalmente `QGraphicsScene` y `QGraphicsObject` para representar el tablero y las cartas del juego. Durante el diseño se buscó mantener una distribución clara de los elementos en pantalla, permitiendo que el usuario identifique rápidamente sus cartas, las cartas del oponente y la zona de combate. Además, se incorporaron animaciones y efectos visuales con el objetivo de mejorar la experiencia de uso sin trasladar lógica del juego hacia la interfaz.

Distribución:

La ventana del juego fue divida en dos secciones donde se utilizó un widget específico para representar donde irían ubicados el mensaje de estados de lo que sucede en la partida. Este widget fue nombrado como widgetHistorial y contienen un label de texto y un textEdit que nos transmite que ocurre en cada turno. Este historial se encuentra ubicado a la izquierda y ocupa un ⅓ de la ventana de juego.

Del lado derecho de la ventana se encuentra el otro widget utilizado para representar el juego y fue llamado widgetJuego que contiene un bloque de graphicsview el cual representa la mesa de juego donde se va a desarrollar absolutamente todo y la clase encargada de esto es la clase **CartaGraphicsItem**. En esta mesa las cartas del jugador humano serán ubicadas en la parte inferior mientras que las cartas del contrincanteIA serán ubicadas en la parte superior, ambos con un total de 5 cartas, en la parte central se desarrollará el combate entre dos cartas elegidas por ambos jugadores y del lado derecho en el centrado verticalmente se encontrará el botón de atacar.

Diseño de las cartas:

Las cartas fueron construidas por varias imágenes separando el marco del icono o imagen que las caracteriza. También cabe aclarar que para dar un efecto de iluminación cuando una carta está por ser seleccionada se utiliza una imagen distinta que es la que se usa al dar el efecto de hover.

CONCEPTOS SOBRE POO

Abstracción

Unos ejemplos presentes en el diagrama de abstracción, se presentan en las clases como Jugador y ContrincanteIA representan conceptos generales, definen estructuras comunes para sus clases derivadas. Esto permite modelar el sistema en niveles más generales sin depender de implementaciones concretas, ya que cada una se enfoca únicamente en implementar los detalles específicos de su comportamiento

Encapsulamiento

Una parte del encapsulamiento, se encuentra en la clase Elemento que administra internamente atributos como la vida y solo permite alterarlos mediante operaciones específicas relacionadas con el combate, en este caso se modificara la vida ante el recibimiento de daño

Herencia

Como se vienen presentando distintos conceptos, la herencia también se encuentra presente en el diagrama, particularmente en la clase abstracta Jugador ,esta misma, define

atributos y comportamientos generales compartidos por todos los tipos de jugadores del sistema. A partir de ella derivan clases como JugadorHumano y ContrincanteIA. A su vez, la clase ContrincanteIA funciona como una base para las distintas implementaciones de inteligencia artificial, como IAAleatoria, IAEstrategica y SuperIA.

Polimorfismo

Tal como la herencia se hace presente, el polimorfismo está incluido en el método seleccionarElemento() este se encuentra definido en las clases base y es implementado de forma diferente según el tipo concreto de jugador o inteligencia artificial. Esto permite que el sistema trabaje de manera general con referencias de tipo Jugador o ContrincanteIA, mientras que cada objeto ejecuta internamente su propia lógica de selección

Composición

Por última instancia, se presenta la composición en la clase Jugador contiene una colección de objetos Elemento, mientras que la clase Juego posee una instancia de SistemaCombate. Estas relaciones indican que ciertos componentes forman parte estructural de otros objetos dentro del sistema.
Esta separación de responsabilidades nos ayuda ,ya que, cada componente puede especializarse en una función concreta y colaborar con otros objetos para cumplir el comportamiento general del juego.

Escalabilidad

Uno de los principales aspectos para la escalabilidad se encuentra en la jerarquía de clases relacionada con los jugadores y las inteligencias artificiales.

La existencia de clases abstractas como **Jugador** y **ContrincanteIA** permite definir comportamientos generales, mientras que las demás clases implementan comportamientos específicos.Por lo tanto, el sistema puede ampliarse fácilmente incorporando nuevos tipos de inteligencia artificial o nuevas estrategias de juego.

La implementación del polimorfismo también contribuye directamente a la escalabilidad del sistema, este mismo permite que el sistema trabaje de forma general con los distintos objetos presentes sin depender de implementaciones específicas. Al mismo tiempo, la separación de responsabilidades entre las clases, esta división modular en las clases reduce el acoplamiento entre componentes y facilita realizar cambios en una parte del sistema sin afectar completamente a las demás. esta separación nos ayudaría si se desea agregar o modificar reglas al juego, en los combates o también incorporar nuevas dinámicas

RESTRICCIONES TÉCNICAS

Si bien hemos estado hablando de las partes técnicas y lógicas que tiene nuestro sistema, queremos ahora introducir las restricciones que tiene el mismo. Como todo juego, hay formas a implementar al momento de crear/programar el juego, a sí mismo, estas formas traen de la mano restricciones que debemos tener en cuenta para que la estructura, lógica y demás de nuestro sistema sea sostenible. También teniendo en cuenta que el diseño mismo posee ciertas restricciones técnicas que podrían afectar su mantenimiento y evolución a medida que aumente la complejidad del proyecto. Estas restricciones no se consideran como errores sino como instancias que podrían presentar dificultades en el futuro del proyecto, como puede ser la idea de una amplificación, por ejemplo.

En base a esto, a continuación nombramos restricciones que notamos en nuestro propio sistema:

1. La restricción que se encuentra en el apartado del combate, más a profundidad en la gestión de relaciones entre los diferentes tipos de elemento. Con la tabla que representa las ventajas y desventajas de cada uno de los elementos. A medida que se agregan más, aumenta la cantidad de relaciones (contra quien tiene ventaja y contra quienes no) que se deben administrar. Esto provoca que el mantenimiento de la lógica del apartado del combate se vuelva aún más complejo
.

2. La restricción relacionada a la estructura de las IAs, la cual podría convertirse en una limitación si el juego aumenta su complejidad. Las Inteligencias Artificiales presentes en el diagrama poseen comportamientos relativamente simples enfocados en seleccionar elementos para combatir, pero si en el futuro se incorporan nuevas mecánicas, habilidades especiales, estados alterados o estrategias avanzadas,entre otros, la lógica interna de estas clases podría crecer significativamente, aumentando la dificultad de mantenimiento nuevamente.